

Exam Preparation Assistance



Unit 4.pdf

PDF

Unit 5.pdf

PDF

I am preparing for my end semester exams I need to give me good answer to the questions I am about to give you. I have attached my syllabus PDFs for your reference

Good. I've gone through your syllabus (Unit IV and Unit V). I'll give you **exam-ready answers**. You just send questions one by one.

I'll keep answers:

- Structured (definition → points → conclusion)
- Easy to write in exam
- Directly aligned with your PDFs (so you score marks, not just "understand")

What I've extracted from your syllabus:

From Unit IV

Unit 4

- User Stories (definition, characteristics, INVEST, 3Cs)
- Writing user stories
- Advantages & disadvantages
- Backlog management
- Feature Driven Development (FDD)
- Agile risk management

From Unit V

Unit 5

- Agile metrics (velocity, lead time, cycle time)
- Agile control parameters
- Risk management (ROAM, RAID)
- Configuration management
- DSDM principles
- Refactoring
- Continuous Integration (CI)

How we'll proceed (this is important)

You send a question → I give:

- **5-10 mark answer (exam-ready)**
- Bullet + paragraph mix
- Keywords highlighted (for marks)
- Optional: short diagram explanation if needed

Example (so you know what you'll get)

If you ask: *"Explain User Stories"*

I'll give:

- Definition
- Characteristics
- Format
- INVEST principle
- 3C's
- Advantages
- Conclusion

Exactly what examiner expects.

Now stop overthinking.

👉 Send your **first question**.

Sources



First write the question then it's answer for each question:--

Define User Story.

List any 5 characteristics of user stories.

What is the Role-Feature-Benefit format of user stories?

What are acceptance criteria?

Define INVEST principle.

What does "Independent" mean in INVEST?

What are the 3 C's of user stories?

What is Backlog Management?

Difference between Product Backlog and Sprint Backlog.

What is Backlog Refinement (Grooming)?

Define MoSCoW prioritization.

What is RICE model?

What is Feature Driven Development (FDD)?

Give one difference between FDD and Scrum.
What is meant by feature in FDD?
What are the 5 activities in FDD?
What is Agile Risk Management?

Here are **clean, exam-ready answers**. Write exactly like this in your paper.

1. Define User Story.

A **user story** is a short, simple description of a feature or functionality written from the perspective of an end user. It captures what the user needs and why, helping the development team understand requirements clearly.

Unit 4

2. List any 5 characteristics of user stories.

1. **User-centric** – Focuses on user needs.
2. **Simple** – Short and easy to understand.
3. **Independent** – Can stand alone.
4. **Negotiable** – Can be modified through discussion.
5. **Testable** – Can be verified using acceptance criteria.

Unit 4

3. What is the Role-Feature-Benefit format of user stories?

User stories follow this format:

"As a [type of user], I want [an action], so that [a benefit/reason]."

Unit 4

4. What are acceptance criteria?

Acceptance criteria are **specific conditions or requirements** that must be satisfied for a user story to be considered complete. They define when the feature is "done" and ensure it meets user expectations.

Unit 4

5. Define INVEST principle.

The **INVEST principle** is a guideline used to evaluate the quality of user stories. It ensures that stories are well-structured, clear, and effective for development.

Unit 4

6. What does "Independent" mean in INVEST?

"Independent" means that a user story should **not depend on other stories** and can be developed and delivered separately.

Unit 4

7. What are the 3 C's of user stories?

1. **Card** – The written user story.
2. **Conversation** – Discussion to clarify requirements.
3. **Confirmation** – Acceptance criteria to validate the story.

Unit 4

8. What is Backlog Management?

Backlog management is the **continuous process of creating, refining, and prioritizing tasks (user stories)** to ensure the team focuses on high-value features during development.

Unit 4

9. Difference between Product Backlog and Sprint Backlog.

Product Backlog	Sprint Backlog
Master list of all features	Subset selected for a sprint
Managed by Product Owner	Managed by Development Team
Dynamic and continuously updated	Fixed during the sprint
Contains long-term goals	Contains short-term tasks

10. What is Backlog Refinement (Grooming)?

Backlog refinement is the process of **reviewing, updating, and breaking down backlog items**, adding details, and estimating effort to keep the backlog clear and actionable.

Unit 4

11. Define MoSCoW prioritization.

MoSCoW is a prioritization technique that classifies requirements into:

- **Must-have**
- **Should-have**
- **Could-have**
- **Won't-have**

Unit 4

12. What is RICE model?

RICE is a prioritization framework based on:

- **Reach**
- **Impact**
- **Confidence**

- **Effort**

It helps rank features based on value and feasibility.

Unit 4

13. What is Feature Driven Development (FDD)?

Feature Driven Development (FDD) is an **Agile methodology** that focuses on building software by developing small, client-valued features in an iterative and incremental manner.

Unit 4

14. Give one difference between FDD and Scrum.

- **FDD** focuses on delivering features frequently.
- **Scrum** focuses on delivering work in fixed time iterations (sprints).

Unit 4

15. What is meant by feature in FDD?

A feature in FDD is a **small, client-valued function** that delivers a specific piece of functionality and provides tangible value to the user.

Unit 4

16. What are the 5 activities in FDD?

1. Develop overall model
2. Build feature list
3. Plan by feature
4. Design by feature
5. Build by feature

Unit 4

17. What is Agile Risk Management?

Agile risk management is a **continuous and iterative process** of identifying, assessing, and mitigating risks throughout development using frequent feedback and incremental delivery.

Unit 4

If you want, next step I can:

- Turn these into **1-page revision notes**, or
- Start giving **long answers (10-mark level)**

Send the next set.

Sources



Explain characteristics of user stories with examples.
 Explain INVEST principle in detail.
 Explain the 3 C's of user stories with examples.
 Write and explain the format of a user story.
 Explain advantages of user stories.
 Explain disadvantages of user stories.
 Explain working with user stories (lifecycle).
 Explain Backlog Management process.
 Explain prioritization techniques (MoSCoW vs RICE).
 Explain FDD methodology and its importance.
 Compare FDD vs Scrum in detail.
 Explain steps in Feature Driven Development.
 Explain Agile risk management process.

Write these answers as-is in your exam. Clean, structured, and marks-oriented.

1. Explain characteristics of user stories with examples.

User stories have the following key characteristics:

1. **User-centric**

Focus on user needs.

Example: As a customer, I want to track my order.

2. **Simple**

Short and easy to understand.

Example: As a user, I want to reset my password.

3. **Independent**

Can be developed separately.

Example: Login feature independent of payment feature.

4. **Negotiable**

Can be discussed and modified.

Example: UI design of dashboard can change after feedback.

5. **Valuable**

Provides value to the user.

Example: Online payment saves time.

6. **Estimable**

Effort can be estimated.

Example: Search feature estimated as 3 story points.

7. **Testable**

Can be verified with criteria.

Example: User should receive confirmation email.

Conclusion: These characteristics ensure clarity, flexibility, and user satisfaction.

Unit 4

2. Explain INVEST principle in detail.

INVEST defines qualities of a good user story:

- **I – Independent:** No dependency on other stories.
- **N – Negotiable:** Open for discussion and refinement.
- **V – Valuable:** Must deliver value to user/customer.
- **E – Estimable:** Effort can be calculated.
- **S – Small:** Should be simple and manageable.
- **T – Testable:** Must have clear acceptance criteria.

Example:

“As a user, I want to receive OTP so that I can securely log in.”

Conclusion: INVEST ensures high-quality and implementable user stories.

Unit 4

3. Explain the 3 C’s of user stories with examples.

1. Card

Written description of story.

Example: “As a user, I want to login.”

2. Conversation

Discussion for clarity.

Example: Developer asks about password rules.

3. Confirmation

Acceptance criteria to verify.

Example: Login works with valid credentials only.

Conclusion: 3 C’s ensure clarity and correctness of requirements.

Unit 4

4. Write and explain the format of a user story.

Format:

“As a [user], I want [action], so that [benefit].”

- **User:** Who needs it
- **Action:** What they want
- **Benefit:** Why they want it

Example:

“As a customer, I want to add items to cart so that I can purchase them later.”

Conclusion: This format ensures clarity and purpose-driven development.

Unit 4

5. Explain advantages of user stories.

1. Focus on user needs
2. Easy to understand
3. Flexible and adaptable
4. Helps in prioritization
5. Improves collaboration
6. Easy estimation and tracking
7. Reduces scope creep

Conclusion: User stories improve communication, flexibility, and product quality.

Unit 4

6. Explain disadvantages of user stories.

1. Lack of detailed requirements
2. Not suitable for complex projects
3. Can be subjective
4. May miss some requirements
5. Not suitable for technical-heavy systems
6. Weak in non-functional requirements

Conclusion: Needs proper refinement to avoid gaps.

Unit 4

7. Explain working with user stories (lifecycle).

1. Write user story
2. Review and refine
3. Add to backlog
4. Prioritize stories
5. Estimate effort
6. Develop during sprint
7. Test using acceptance criteria
8. Deliver and get feedback

Conclusion: It is an iterative cycle ensuring continuous improvement.

Unit 4

8. Explain Backlog Management process.

Backlog management involves:

1. **Create backlog** (list of features)
2. **Refinement (Grooming)**
3. **Prioritization** (MoSCoW, RICE)
4. **Estimation**
5. **Sprint planning**
6. **Continuous updates**

Conclusion: Ensures focus on high-value tasks and efficient development.

Unit 4

9. Explain prioritization techniques (MoSCoW vs RICE).

MoSCoW

- Must-have
- Should-have
- Could-have
- Won't-have

RICE

- Reach
- Impact
- Confidence
- Effort

Difference

- MoSCoW is **qualitative**
- RICE is **quantitative**

Conclusion: Both help prioritize features effectively.

Unit 4

10. Explain FDD methodology and its importance.

FDD is an Agile method focusing on building features.

Characteristics:

- Client-centric
- Iterative
- Incremental

Importance:

- Faster delivery
- Better progress tracking
- Reduces rework
- Improves quality

Conclusion: Ensures frequent and reliable delivery of features.

Unit 4

11. Compare FDD vs Scrum in detail.

Feature	FDD	Scrum
Focus	Features	Sprints
Delivery	Every 2–10 days	2–4 weeks
Approach	Feature-based	Time-based
Meetings	Less frequent	Daily stand-ups
Documentation	More	Less

Conclusion: FDD is feature-focused, Scrum is time-boxed.

Unit 4

12. Explain steps in Feature Driven Development.

1. Develop overall model
2. Build feature list
3. Plan by feature
4. Design by feature
5. Build by feature

Conclusion: Provides a structured and efficient development process.

Unit 4

13. Explain Agile risk management process.

Agile risk management is continuous:

1. Identify risks
2. Assess risks
3. Prioritize risks
4. Mitigate risks
5. Monitor continuously
6. Use feedback and iteration

Techniques:

- Daily meetings
- Incremental delivery
- Stakeholder feedback

Conclusion: Reduces risk through early detection and continuous improvement.

Unit 4

If you want to **level up your prep**, next I can:

- Turn all of this into **last-day revision sheets**
- Or give **expected exam questions + answers** (very high probability)

Send next set.

Sources



👉 "User stories are the backbone of Agile development."

Explain with:

Characteristics

INVEST principle

3C's

Advantages & disadvantages

👉 Explain Backlog Management in Agile with:

Product backlog

Sprint backlog

Refinement

Prioritization techniques

👉 Explain Feature Driven Development (FDD) in detail:

Definition

Process steps

Differences with Scrum

Advantages

👉 Explain Agile Risk Management and how Agile reduces project risks.

Write these as **10–15 mark answers**. Structured, to-the-point, and scoring.

1. "User stories are the backbone of Agile development." Explain.

User stories are fundamental in Agile because they **capture user requirements in a simple, flexible, and value-driven way**, guiding development, planning, and delivery.

(a) Characteristics of User Stories

1. **User-centric** – Focus on user needs
2. **Simple** – Short and easy to understand
3. **Independent** – Can be developed separately
4. **Negotiable** – Flexible and open to change
5. **Valuable** – Deliver business value
6. **Estimable** – Effort can be estimated
7. **Testable** – Can be validated with criteria

Example:

"As a user, I want to reset my password so that I can regain access."

(b) INVEST Principle

A good user story follows INVEST:

- **Independent** – No dependency
 - **Negotiable** – Open for discussion
 - **Valuable** – Useful to user
 - **Estimable** – Can estimate effort
 - **Small** – Easy to implement
 - **Testable** – Has acceptance criteria
-

(c) 3 C's of User Stories

1. **Card** – Written story
 2. **Conversation** – Discussion for clarity
 3. **Confirmation** – Acceptance criteria
-

(d) Advantages of User Stories

- Focus on user needs
- Easy to understand
- Flexible to changes
- Improves collaboration
- Helps prioritization

- Reduces scope creep

(e) Disadvantages of User Stories

- Lack of detailed documentation
- Not suitable for complex systems
- May miss requirements
- Subjective interpretation
- Weak for non-functional requirements

Conclusion

User stories act as the **core unit of Agile development**, enabling continuous delivery of user value and adaptability.

Unit 4

2. Explain Backlog Management in Agile

Backlog management is the **continuous process of managing, refining, and prioritizing tasks** to ensure development focuses on high-value features.

(a) Product Backlog

- Master list of all features, fixes, improvements
- Managed by Product Owner
- Dynamic and continuously updated

(b) Sprint Backlog

- Subset of product backlog
- Selected for current sprint
- Managed by development team

(c) Backlog Refinement (Grooming)

- Breaking large tasks into smaller ones
- Adding details and acceptance criteria
- Estimating effort
- Removing outdated items

(d) Prioritization Techniques

MoSCoW

- Must-have
- Should-have
- Could-have
- Won't-have

RICE

- Reach
- Impact
- Confidence
- Effort

Difference:

MoSCoW is qualitative, RICE is data-driven.

Conclusion

Effective backlog management ensures **better planning, adaptability, and delivery of value.**

Unit 4

3. Explain Feature Driven Development (FDD) in detail

(a) Definition

FDD is an Agile methodology that focuses on **developing small, client-valued features in an iterative and incremental manner.**

(b) Process Steps in FDD

1. Develop overall model
2. Build feature list
3. Plan by feature
4. Design by feature
5. Build by feature

(c) Difference between FDD and Scrum

Aspect	FDD	Scrum
Focus	Features	Sprints
Delivery	Frequent feature delivery	Time-boxed delivery
Meetings	Less frequent	Daily stand-ups

Aspect	FDD	Scrum
Approach	Feature-driven	Iteration-driven
Documentation	More	Less

(d) Advantages of FDD

- Faster delivery of features
- Better progress tracking
- Reduced rework
- High customer satisfaction
- Suitable for large projects

Conclusion

FDD provides a **structured and feature-focused approach**, ensuring continuous delivery of valuable functionality.

Unit 4

4. Explain Agile Risk Management and how Agile reduces project risks

(a) Agile Risk Management

Agile risk management is a **continuous and proactive process** of identifying, assessing, and mitigating risks throughout development.

(b) Risk Management Process

1. Identify risks
2. Assess impact and probability
3. Prioritize risks
4. Mitigate risks
5. Monitor continuously
6. Adapt using feedback

(c) How Agile Reduces Risks

- **Frequent iterations** → early detection of issues
- **Continuous feedback** → quick correction

- **Small increments** → limited risk exposure
- **Daily communication** → transparency
- **Testing in every sprint** → better quality
- **Stakeholder involvement** → correct requirements

Conclusion

Agile minimizes risks by **detecting problems early, adapting quickly, and delivering continuously**, ensuring project success.

Unit 4

If you want to seriously maximize marks, next I can:

- Convert all this into **1-night revision cheat sheet**
- Or give **most expected long questions + answers** (exam prediction)

Your move.

Sources



Write 3 user stories for an e-commerce app using correct format.
 Convert given requirement into user stories.
 Identify whether a user story follows INVEST or not.
 Given backlog → prioritize using MoSCoW.
 Given scenario → identify risks and mitigation.
 Break a large requirement into smaller user stories.

These are **application-type answers**. Write clearly with examples and steps. Examiner gives marks for logic + format.

1. Write 3 user stories for an e-commerce app using correct format

Format: *As a [user], I want [action], so that [benefit]*

1. As a **customer**, I want to **search products**, so that I can **find items easily**.
2. As a **customer**, I want to **add items to cart**, so that I can **purchase multiple products together**.
3. As a **customer**, I want to **track my order**, so that I can **know delivery status**.

2. Convert given requirement into user stories

Requirement:

"Users should be able to login and reset password."

Converted User Stories:

1. As a **user**, I want to **log in using email and password**, so that I can **access my account**.
2. As a **user**, I want to **reset my password**, so that I can **recover access if I forget it**.

👉 Rule: **Break requirement** → **multiple small user stories**

3. Identify whether a user story follows INVEST or not

User Story:

"As a user, I want a full website with login, payment, and tracking."

Analysis (INVEST):

- Independent ❌ (depends on many features)
- Negotiable ✓
- Valuable ✓
- Estimable ❌ (too big)
- Small ❌ (very large)
- Testable ❌

Conclusion:

❌ **Does NOT follow INVEST**

👉 Correct version: break into smaller stories.

4. Given backlog → prioritize using MoSCoW

Backlog Items:

- Login system
- Payment gateway
- Dark mode
- Product recommendations

Prioritization:

- **Must-have:** Login system, Payment gateway
- **Should-have:** Product recommendations

- **Could-have:** Dark mode
- **Won't-have:** (none in this case or future feature)

👉 Logic: prioritize **core functionality first**

5. Given scenario → identify risks and mitigation

Scenario:

Developing an e-commerce app with new payment integration.

Risks & Mitigation:

1. **Technical risk:** Payment API may fail
→ *Mitigation:* Use sandbox testing and backup gateway
 2. **Requirement risk:** Changing client needs
→ *Mitigation:* Frequent feedback and sprint reviews
 3. **Security risk:** Data breach
→ *Mitigation:* Use encryption and secure APIs
 4. **Time risk:** Delays in integration
→ *Mitigation:* Break into small tasks and monitor progress
-

6. Break a large requirement into smaller user stories

Large Requirement:

"Build an online shopping system."

Broken User Stories:

1. As a user, I want to **register**, so that I can create an account
2. As a user, I want to **log in**, so that I can access my account
3. As a user, I want to **browse products**, so that I can choose items
4. As a user, I want to **add to cart**, so that I can purchase items
5. As a user, I want to **make payment**, so that I can complete purchase
6. As a user, I want to **track orders**, so that I know delivery status

👉 Key idea:

- Break into **small, independent, testable stories**
-

Final Tip (Important for Exam)

Whenever you see:

- "Write user story" → follow exact format
- "Convert requirement" → split into multiple stories
- "INVEST" → check size + independence
- "MoSCoW" → prioritize core first
- "Break requirement" → think like real app features

If you want, I can next:

- Give **20+ practice questions like this (high probability)**
- Or do a **mock test + evaluate your answers brutally**

Say the word.



What is Agile Measurement?

Define:

Velocity

Cycle Time

Lead Time

What are escaped defects?

What is Agile Control?

List 7 control parameters in Agile.

What is meant by:

Scope (in Agile)

Time (in Agile)

What is Sprint Burndown Chart?

What is Daily Standup?

What is Kanban Board?

What is Retrospective?

What is RAID log?

What is ROAM technique?

Define Configuration Management (CM).

What is Refactoring?

What is Continuous Integration (CI)?

List benefits of CI.

Write these exactly like this in your exam. Clean definitions + keywords.

1. What is Agile Measurement?

Agile measurement is the process of **tracking progress, quality, and value delivery** in Agile projects using metrics like velocity, cycle time, and lead time to improve efficiency and customer satisfaction.

Unit 5

2. Define Velocity

Velocity is the **amount of work completed by a team in a single sprint**, usually measured in story points, used to predict future performance.

Unit 5

3. Define Cycle Time

Cycle time is the **time taken to complete a task from start to finish** once work begins.

Unit 5

4. Define Lead Time

Lead time is the **total time from when a request is made until it is completed and delivered**.

Unit 5

5. What are escaped defects?

Escaped defects are **bugs that are not detected during development/testing and are found after release in production**.

Unit 5

6. What is Agile Control?

Agile control is the process of **managing project parameters through iterative development, continuous feedback, and collaboration**, ensuring flexibility and quality.

Unit 5

7. List 7 control parameters in Agile

1. Scope
2. Time
3. Cost
4. Quality
5. Risk
6. Resources
7. Benefits (Value)

Unit 5

8. What is meant by Scope (in Agile)?

Scope in Agile is **flexible and changeable**, managed through a prioritized backlog instead of fixed requirements.

Unit 5

9. What is meant by Time (in Agile)?

Time in Agile is **fixed using time-boxed iterations (sprints)**, usually lasting 1–4 weeks.

Unit 5

10. What is Sprint Burndown Chart?

A Sprint Burndown Chart is a **graph showing remaining work versus time in a sprint**, used to track progress and ensure timely completion.

Unit 5

11. What is Daily Standup?

Daily Standup is a **short daily meeting where team members share progress, plans, and blockers** to ensure coordination.

Unit 5

12. What is Kanban Board?

A Kanban board is a **visual tool that shows workflow stages (To Do, In Progress, Done)** and helps manage tasks efficiently.

Unit 5

13. What is Retrospective?

A Retrospective is a **meeting held after each sprint to review what went well, what didn't, and how to improve.**

Unit 5

14. What is RAID log?

RAID log is a tool used to track:

- **Risks**
- **Assumptions**
- **Issues**
- **Dependencies**

Unit 5

15. What is ROAM technique?

ROAM categorizes risks into:

- **Resolved**
- **Owned**
- **Accepted**

- **Mitigated**

Unit 5

16. Define Configuration Management (CM)

Configuration Management is the process of **managing and tracking changes in project assets like code, documents, and systems to maintain consistency and control.**

Unit 5

17. What is Refactoring?

Refactoring is the process of **improving the internal structure of code without changing its external behavior**, enhancing readability and maintainability.

Unit 5

18. What is Continuous Integration (CI)?

Continuous Integration is a practice where **developers frequently merge code into a shared repository, triggering automated builds and tests** to ensure code quality.

Unit 5

19. List benefits of CI

1. Early bug detection
2. Reduced integration issues
3. Faster development cycles
4. Improved code quality
5. Increased productivity
6. Faster feedback to developers

Unit 5

If you want to finish this prep properly, next I can:

- Give **full Unit 4 + 5 revision in 30 minutes**
- Or **predict your exam paper with answers**

Don't slow down now.

Sources



Explain Agile metrics in detail.
 Explain Agile control parameters.
 Explain difference between Agile and traditional control.
 Explain risk management in Agile.
 Explain Configuration Management in Agile.

Explain challenges of CM in Agile.
 Explain best practices for CM in Agile.
 Explain DSDM principles.
 Explain Refactoring and its importance.
 Explain Continuous Integration process.

These are **long-answer (10–15 marks)**. Write in headings + points. Don't write essays.

1. Explain Agile metrics in detail

Agile metrics are used to **measure performance, flow, quality, and value delivery** instead of just output.

Key Agile Metrics

1. Velocity

Amount of work completed in a sprint (story points).
 Helps in planning future sprints.

2. Cycle Time

Time taken to complete a task once started.
 Indicates development speed.

3. Lead Time

Total time from request to delivery.
 Measures customer wait time.

4. Escaped Defects

Bugs found after release.
 Indicates quality of testing.

Importance

- Improves productivity
- Helps in forecasting
- Reduces technical debt
- Enhances customer satisfaction

Conclusion: Agile metrics focus on **continuous improvement and value delivery**.

Unit 5

2. Explain Agile control parameters

Agile control manages projects through flexibility and continuous feedback.

7 Control Parameters

1. **Scope** – Flexible, managed via backlog

2. **Time** – Fixed (sprints)
3. **Cost** – Depends on iterations
4. **Quality** – Maintained continuously
5. **Risk** – Managed iteratively
6. **Resources** – Stable, cross-functional teams
7. **Benefits (Value)** – Focus on delivering value first

Conclusion: Agile balances flexibility with control.

Unit 5

3. Explain difference between Agile and traditional control

Aspect	Agile Control	Traditional Control
Scope	Flexible	Fixed
Time	Fixed	Flexible
Approach	Iterative	Sequential
Feedback	Continuous	Limited
Risk	Managed early	Managed later
Customer involvement	High	Low

Conclusion: Agile is adaptive, traditional is rigid.

Unit 5

4. Explain risk management in Agile

Agile risk management is **continuous and proactive**.

Process

1. Identify risks regularly
2. Assess impact
3. Prioritize risks
4. Mitigate early
5. Monitor continuously

Techniques

- Daily stand-ups
- Sprint reviews
- RAID log
- ROAM technique

Key Features

- Small iterations reduce impact
- Early feedback reduces uncertainty
- Transparency improves decision-making

Conclusion: Agile reduces risk through **continuous monitoring and fast adaptation**.

Unit 5

5. Explain Configuration Management (CM) in Agile

Configuration Management is the process of **tracking and controlling changes in software components** like code, documents, and systems.

In Agile

- Supports fast iterations
- Ensures consistency
- Maintains version control

Examples

- Git repositories
- CI/CD pipelines

Conclusion: CM ensures stability in a fast-changing Agile environment.

Unit 5

6. Explain challenges of CM in Agile

1. Frequent updates and iterations
2. Version control complexity
3. Collaboration across teams
4. Balancing documentation with agility
5. Risk of inconsistencies

Conclusion: Agile speed can create chaos without proper CM.

Unit 5

7. Explain best practices for CM in Agile

1. Use version control tools (Git)
2. Automate builds and testing (CI/CD)
3. Maintain lightweight documentation
4. Use tagging and release tracking

5. Ensure team communication
6. Use tools like Jira, Azure DevOps

Conclusion: Proper practices ensure control without slowing Agile speed.

Unit 5

8. Explain DSDM principles

DSDM (Dynamic Systems Development Method) is an Agile framework with **8 principles**:

1. Focus on business need
2. Deliver on time
3. Collaborate
4. Never compromise quality
5. Build incrementally
6. Develop iteratively
7. Communicate continuously
8. Demonstrate control

Conclusion: DSDM ensures **timely delivery with quality and collaboration**.

Unit 5

9. Explain Refactoring and its importance

Refactoring is **improving code structure without changing functionality**.

Importance

- Improves readability
- Reduces technical debt
- Enhances maintainability
- Prevents code complexity
- Supports future changes

Example

- Renaming variables
- Simplifying logic

Conclusion: Refactoring keeps code clean and scalable.

Unit 5

10. Explain Continuous Integration process

Continuous Integration (CI) is the practice of **frequently merging code into a shared repository with automated testing.**

Process

1. Developer writes code
2. Code is committed to repository
3. Automated build starts
4. Automated tests run
5. Errors are reported immediately
6. Code is integrated successfully

Benefits

- Early bug detection
- Faster development
- Reduced integration issues
- Improved code quality

Conclusion: CI ensures **fast, reliable, and high-quality software delivery.**

Unit 5

If you want to go all-in:

- I can compress ALL of Unit 4 + 5 into **5 pages (ultimate revision)**
- Or take a **mock test and grade you brutally**

Say it.

Sources



👉 Explain Agile Measurement and Control with all metrics and tools.

👉 Explain Agile Risk Management with:

Continuous identification
ROAM
RAID
Mitigation strategies

👉 Explain Configuration Management in Agile:

Challenges
Best practices
Tools

👉 Explain Continuous Integration (CI) in Agile:

Process
Benefits
Tools

👉 Explain Refactoring in Agile:

Techniques
Benefits
Challenges

Write these as **15-mark answers**. Use headings, keywords, and examples.

1. Explain Agile Measurement and Control with all metrics and tools

Agile measurement and control focus on **tracking progress, quality, and value delivery** while maintaining flexibility and continuous improvement.

(a) Agile Measurement

Agile measurement evaluates performance using key metrics:

Key Metrics

1. **Velocity** – Work completed in a sprint (story points)
2. **Cycle Time** – Time taken to complete a task
3. **Lead Time** – Time from request to delivery
4. **Escaped Defects** – Bugs found after release

Importance

- Improves productivity
- Helps in forecasting
- Ensures quality
- Enhances customer satisfaction

(b) Agile Control

Agile control manages projects through iterative feedback.

Control Parameters

- Scope (flexible)
 - Time (fixed sprints)
 - Cost
 - Quality
 - Risk
 - Resources
 - Benefits (value)
-

(c) Tools for Measurement and Control

- **Sprint Burndown Chart** – Tracks remaining work
 - **Velocity Chart** – Tracks team performance
 - **Kanban Board** – Visualizes workflow
 - **Daily Standup** – Tracks progress and blockers
 - **Retrospective** – Improves processes
-

Conclusion

Agile measurement and control ensure **continuous monitoring, adaptability, and value delivery**.

Unit 5

2. Explain Agile Risk Management

Agile risk management is a **continuous, proactive process** integrated into development.

(a) Continuous Identification

- Risks identified regularly (daily standups, sprint planning)
 - Not a one-time activity
-

(b) ROAM Technique

- **Resolved** – Risk eliminated
 - **Owned** – Assigned to someone
 - **Accepted** – Acknowledged
 - **Mitigated** – Reduced impact
-

(c) RAID Log

Tracks:

- **Risks**
 - **Assumptions**
 - **Issues**
 - **Dependencies**
-

(d) Mitigation Strategies

- Break work into small increments
 - Use spikes for technical uncertainty
 - Frequent testing and feedback
 - Early stakeholder involvement
-

Conclusion

Agile minimizes risks through **early detection, continuous monitoring, and quick adaptation.**

Unit 5

3. Explain Configuration Management in Agile

Configuration Management (CM) ensures **control and consistency of project artifacts** like code and documents.

(a) Challenges in Agile CM

1. Frequent changes and iterations
 2. Version control complexity
 3. Cross-team collaboration issues
 4. Limited documentation
 5. Risk of inconsistency
-

(b) Best Practices

1. Use version control systems (Git)
2. Automate builds and testing
3. Maintain lightweight documentation
4. Use tagging and release tracking

5. Ensure team communication

(c) Tools

- Git, Mercurial
 - Jira, Azure DevOps
 - CI/CD tools (Jenkins, GitLab CI)
-

Conclusion

CM ensures **stability and traceability in fast-paced Agile environments.**

Unit 5

4. Explain Continuous Integration (CI) in Agile

Continuous Integration is a practice where **developers frequently integrate code with automated testing and builds.**

(a) CI Process

1. Developer writes code
 2. Code is committed to repository
 3. Automated build is triggered
 4. Automated tests run
 5. Errors are detected early
 6. Code is integrated into main branch
-

(b) Benefits

- Early bug detection
 - Reduced integration issues
 - Faster development cycles
 - Improved code quality
 - Immediate feedback
-

(c) Tools

- Jenkins
- GitLab CI/CD

- CircleCI
 - TeamCity
-

Conclusion

CI ensures **fast, reliable, and high-quality software delivery**.

Unit 5

5. Explain Refactoring in Agile

Refactoring is the process of **improving internal code structure without changing its functionality**.

(a) Techniques

- Renaming variables
 - Extracting methods
 - Removing duplication
 - Simplifying complex logic
 - Replacing conditional logic
-

(b) Benefits

- Improves code readability
 - Reduces technical debt
 - Enhances maintainability
 - Supports scalability
 - Makes future changes easier
-

(c) Challenges

- Time constraints in sprints
 - Risk of breaking code
 - Requires strong test coverage
 - Balancing new features vs improvements
-

Conclusion

Refactoring ensures **clean, maintainable, and adaptable code** in Agile systems.

Unit 5

If you want to lock this down completely, next I can:

- Give you **last-day revision (super compressed)**
- Or simulate a **real exam paper with solutions**

Don't stop now.

Sources



Scenario-Based / Analytical
 Given sprint data → calculate velocity.
 Identify cycle time vs lead time from scenario.
 Suggest risk mitigation strategy.
 Given code issues → suggest refactoring techniques.
 Given team workflow → identify CI benefits.
 Given project → suggest Agile metrics to track

These are **scenario-based answers**. In exam, show **formula + logic + conclusion**. That's how you score.

1. Given sprint data → calculate velocity

Formula:

Velocity = Total story points completed in a sprint

Example:

Sprint completed stories: 5, 8, 3, 4 story points

Velocity = $5 + 8 + 3 + 4 = 20$ story points

Conclusion:

Velocity helps **predict future sprint capacity**.

2. Identify cycle time vs lead time from scenario

Key Difference:

- **Cycle Time:** Work start → Work complete
- **Lead Time:** Request → Delivery

Example Scenario:

- Request made on Day 1

- Work started on Day 3
- Completed on Day 8
- **Lead Time = Day 1 → Day 8 = 7 days**
- **Cycle Time = Day 3 → Day 8 = 5 days**

Conclusion:

Lead time includes waiting, cycle time does not.

3. Suggest risk mitigation strategy

Scenario: Payment integration may fail

Mitigation Strategies:

- Use backup payment gateway
- Perform sandbox testing
- Break into smaller tasks
- Use spikes for research
- Continuous testing in sprints

Conclusion:

Mitigate early to reduce impact.

4. Given code issues → suggest refactoring techniques

Scenario: Code is complex and repetitive

Refactoring Techniques:

- Extract methods (break large functions)
- Remove duplication
- Rename variables (improve clarity)
- Simplify logic
- Replace conditional logic with better structure

Conclusion:

Refactoring improves maintainability without changing behavior.

5. Given team workflow → identify CI benefits

Scenario: Multiple developers working on same codebase

CI Benefits:

- Early bug detection
- Avoids merge conflicts
- Faster integration
- Continuous feedback
- Improved code quality

Conclusion:

CI ensures smooth collaboration and stable builds.

6. Given project → suggest Agile metrics to track

Scenario: Developing e-commerce app

Recommended Metrics:

- **Velocity** → Track team performance
- **Cycle Time** → Measure development speed
- **Lead Time** → Measure delivery efficiency
- **Escaped Defects** → Ensure quality

Conclusion:

Use metrics that track **speed, quality, and value delivery**.

Exam Hack (Very Important)

Whenever you see scenario:

1. Write **formula/definition**
2. Apply to given data
3. Show **final answer clearly**

That alone gets you 80% marks.

If you want to dominate this paper, next I can:

- Give **full mock exam (with case-based questions)**
- Or make **last-day ultra revision (everything in 20 mins)**